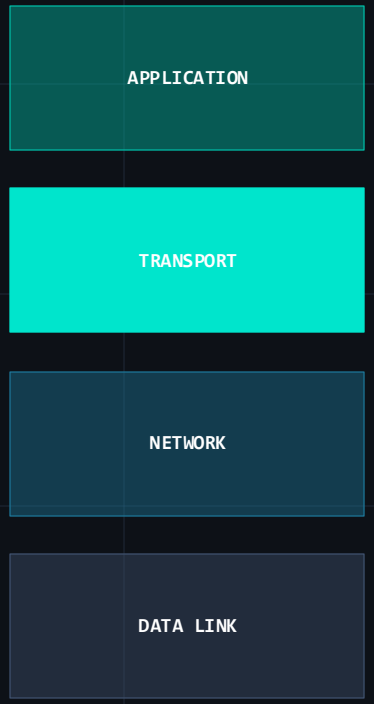
LAB · REȚELE DE CALCULATOARE

More on Network Layer

Docker · Scapy · UDP · IP Spoofing



Docker Image

Recap pt Tema 2

01

Ce este o imagine Docker? = șablon read-only

`CMD / ENTRYPOINT`	Ce rulează la pornire
`COPY — codul`	Fișierele aplicației
`RUN — pip / apt`	Pachete instalate → fiecare instrucțiune din Dockerfile = un layer
`WORKDIR`	Directorul de lucru
`FROM python:3.11-slim`	Imaginea de bază

Cache:
Dacă un layer nu s-a schimbat, Docker îl refolosește. De aceea requirements.txt se copiază ÎNAINTE de cod.

Fiecare instrucțiune = un layer read-only.
Containerul adaugă un layer writable deasupra.

pip install vs apt-get install

pip install

Librării Python

```
pip install flask
pip install scapy
pip install requests
pip install numpy
```

Pachete Python = librării importabile în cod.
PyPI = registrul global.

✓ flask ✓ scapy ✓ requests

apt-get install

Programe de sistem Linux

```
apt-get update
apt-get install -y curl
apt-get install -y wget
apt-get install -y vim
```

Binare de sistem = programe executabile,
nu importabile în Python.

✓ curl ✓ wget ✓ nmap ✓ vim

Dockerfile → Build → Tag → Push

```
FROM python:3.11-slim

WORKDIR /app

RUN pip install flask \
    && apt-get update \
    && apt-get install -y curl

EXPOSE 8001

CMD ["bash"]
```

- Baza - slim = ~50MB vs ~900MB pentru varianta full
- Directorul de lucru - toate comenzile sunt relative la /app
- Instalezi ambele tipuri de pachete într-un singur RUN (mai puține layers)
- EXPOSE = documentație, nu deschide portul! -p face asta la docker run
- CMD bash = containerul stă activ fără o aplicație specifică

1. `docker build -t tema_2 .` → construieste imaginea local
2. `docker run -it tema_2 bash` → verifici că pachetele există
3. `docker tag tema_2 user/tema_2:latest` → tag pentru Docker Hub
4. `docker push user/tema_2:latest` → publici imaginea

Docker Images – comenzi utile

```
# Listezi toate imaginile locale
docker images

# Rulezi interactiv să verifici ce e instalat
docker run -it numele_meu bash

# Din container, verifici că un pachet există:
pip show flask
curl --version
python3 -c "import flask; print(flask.__version__)"
```

De ce poate fi importantă minimizarea numărului de pachete dintr-o imagine?

- Imagini mai mici = download mai rapid
- Suprafață de atac mai mică (mai puține vulnerabilități)

Docker Compose

02

Când ai mai multe containere care trebuie să comunice, să pornești fiecare cu docker run devine complicat. Compose definește toate serviciile într-un singur fișier și le pornește, conectează și configurează automat.

Servicii, Rețele, Volume, depends_on

Serviciu

= un container. Are imagine, rețea, volume, comenzi. Primește un nume, care devine hostname-ul în rețea.

Rețea

= Containere din aceeași rețea se văd direct. Docker oferă DNS intern: 'server' → IP-ul containerului server.

Volume

= folder partajat. ./app:/app face ca folderul local ./app să apară la /app în container. Editezi local (PyCharm, Visual Studio Code etc.), rulezi în container.

depends_on

= ordine de pornire. In configuratia noastra server/client: Clientul pornește după server. Param nu garantează că serverul e gata, doar că a pornit.

Structura docker-compose.yml

```yaml
version: '3'

services:

  server:
    image: tema_2
    tty: true
    networks:
      - app_net
    volumes:
      - ./app:/app

  client:
    image: tema_2
    tty: true
    networks:
      - app_net
    volumes:
      - ./app:/app
    depends_on:
      - server

networks:
  app_net:
    ipam:
      driver: default
      config:
        - subnet: 172.20.0.0/16
          gateway: 172.20.0.1
```

- image: de exemplu, tema_2 = imaginea construită deja
- tty: true = ține containerul viu (fără un proces activ)
- networks: app_net = se conectează la rețeaua virtuală
- volumes: ./app:/app = editezi în PyCharm, rulezi în container
- depends_on = clientul pornește după server
- subnet definit = IP-uri predictibile în rețeaua Docker

DNS intern Docker

```
# Adresa serverului din client:
SERVER_IP = "server"   # ← numele serviciului
```

UDP & TCP

Porturi, headers, sport/dport

03

IP + Port = identificarea exactă a unui proces

172.20.0.2

(adresa blocului)

:9999	server UDP
:8001	Flask HTTP
::53	DNS server
::22	SSH

0 – 1023	Well-known (root necesar)
1024 – 49151	Registered
49152 – 65535	Ephemeral (OS le alege)

sport vs dport

```
dport  =  portul DESTINATIEI
          unde vrei sa ajungi
          tu il alegi, serverul il asculta

sport  =  portul SURSEI
          de unde "esti"
          ales de OS automat (1024-65535)
          sau de tine cu Scapy
```

Serverul vede addr = (ip_sursa, sport)
Dacă sport e ales de OS → aleatoriu la fiecare pachet
Dacă sport e setat de tine → predictibil, fix

UDP = simplu vs TCP = complex

UDP Header — 8 bytes

sport (16)	dport (16)	length (16)	checksum (16)
payload / data			

→ Connectionless, aka fara handshake

→ Fara garantii de livrare

→ Fara ordine garantata

→ Simplu de manipulat cu Scapy

→ Rapid, overhead minim

TCP Header — 20+ bytes

sport (16)	dport (16)
Sequence Number (32)	
Acknowledgment Number (32)	
Flags | Window Size (16)	
Checksum | Urgent (16+16)	

→ Connection-oriented, 3-way handshake

→ Garanteaza livrarea (retransmisii)

→ Garanteaza ordinea

→ Flow control + congestion control

→ Mai complex de manipulat cu Scapy

Lab-ul folosește UDP, mai simplu de manipulat la nivel de pachete

Scapy

04

Tu construiesti fiecare strat manual

Socket normal

```
sock.sendto(b"salut", (dest, 9999))
# OS adauga UDP + IP + Ethernet
# Nu ai control asupra campurilor
```

✗ sport aleatoriu ✗ src = IP real ✗ fara control

Scapy

```
IP(src=..., dst=...) \
  / UDP(sport=..., dport=...) \
  / Raw(b"salut")
# Tu controlezi fiecare camp
```

✓ sport fix ✓ src fals ✓ control total

Operatorul / pt layering

```
# Fiecare / adauga un strat, exact ca modelul OSI
pachet = Ether() / IP(dst="172.20.0.2") / UDP(dport=9999) / Raw(b"mesaj")
#          L2        L3                    L4                payload

# Campurile None sunt calculate automat de Scapy la trimitere:
# IP.src    = IP-ul tau real (daca nu il specifici tu)
# UDP.sport = port efemer aleatoriu
# IP.chksum = calculat din continut
```

IP, UDP, TCP, ICMP, Raw — campuri si inspectare

```
# Layer 3
IP(src="...", dst="...", ttl=64)

#Layer 4 - UDP
UDP(sport=12345, dport=9999)

#Layer 4 - TCP
TCP(sport=..., dport=80, flags="S")  # S = SYN, PA = PSH+ACK
#PSH = flag that indicates the receiving device should deliver
the data to the receiving application as soon as possible

#ICMP (ping)
ICMP(type=8)   # 8=Echo Request, 0=Echo Reply

#Payload
Raw(load=b"bytes bruti")   # sau load="string"
```

```
# Inspectare
pachet.show()
pachet.summary()

# Verifica prezenta unui layer
pachet.haslayer(UDP)    # True / False

# Acceseaza campuri
pachet[IP].src
pachet[UDP].sport
pachet[Raw].load
```

Functii de trimitere

`send(p)`	Layer 3, nu asteapta raspuns
`sr1(p)`	Layer 3, returneaza primul raspuns
`sr(p)`	Layer 3, returneaza toate
`sendp(p)`	Layer 2, cu Ether(), nu asteapta
`srp1(p)`	Layer 2, primul raspuns

Problema loopback:

```
# Daca send() nu merge pe localhost:
conf.L3socket = L3RawSocket
# sau: send(p, iface="eth0")
```

IP Spoofing

Sursa declarata ≠ sursa reala

05

Protocolul IP nu autentifica sursa

Analogia poștei

Trimiți o scrisoare cu o adresă de expeditor falsă pe plic. Poșta o livrează uitându-se DOAR la destinatar. Destinatarul citește expeditorul de pe plic și îl crede , pentru ca nu există mecanism de verificare. IP-ul sursă e adresa de pe plic.

De ce nu poți face asta cu socket normal?

```
# Socket standard — OS pune IP-ul real, nu poti schimba
sock.sendto(b"mesaj", ("dest", 9999))
# → addr pe server: ('IP_REAL_AL_TAU', port)  — mereu

# Scapy — tu controlezi campul src din headerul IP
IP(src="172.20.0.99", dst="dest") / UDP(dport=9999) / Raw(b"mesaj")
# → addr pe server: ('172.20.0.99', port)  — IP fictiv!
```

Ce face serverul:

`data, addr = sock.recvfrom(1024)` → `addr = (ip_din_headerIP, sport_din_headerUDP)` ≠ neaparat realitatea

Experiment

1. **Trimite un pachet normal**
   Porneste server + client. Trimite un pachet fara a specifica src. Noteaza ce apare in addr pe server.

2. **Afla IP-ul real al clientului**
   ip addr show eth0 sau socket.gethostbyname(socket.gethostname()). Retine valoarea.

3. **Adauga src=IP_REAL explicit**
   Pune IP-ul real in IP(src=...). Trimite. S-a schimbat ceva in addr? De ce / de ce nu?

4. **Schimba src cu IP fictiv**
   Alege un IP din acelasi subnet (ex: 172.20.0.99). Pune-l in src. Trimite. Ce apare acum in addr?

5. **Incearca IP din alta retea**
   Alege un IP dintr-un alt subnet (ex: 10.0.0.99). Ce se intampla? Ajunge pachetul? De ce?

De ce conteaza in lumea reala

DDoS Attacks

DNS Flood

Trimiți cereri DNS cu src = victima. Serverul DNS răspunde victimei cu date mult mai mari decât cererea. Victima e inundată.

SYN Flood

Inundezi un server cu SYN-uri cu IP-uri false. Serverul ține conexiuni half-open și epuizează resursele (tabela de conexiuni).

```
sport fix + IP spoof: IP(src=FICTIV, dst=SERVER) / UDP(sport=12345, dport=9999) / Raw(b"mesaj")
```

BCP38

Best Current Practice 38

ISP-urile care implementeaza BCP38 blocheaza pachetele cu IP sursa care nu apartin subnet-ului clientului.

Spoofing-ul functioneaza in retele locale / Docker, dar e blocat pe internet de ISP-urile responsabile.